Laporan Tugas Besar 1

IF2211 Strategi Algoritma
Pemanfaatan Algoritma *Greedy* dalam pembuatan bot permainan Battlecode 2025

Semester II 2025/2026

Disusun oleh Kelompok AcademyOfFineArts

Bernhard Aprillio Pramana	13524074
Moreno Syawali Ganda Sugita	13524096
Nathaniel Christian	13524122

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI i
BAB 1
DESKRIPSI TUGAS 1
BAB II
LANDASAN TEORI 2
2.1. Algoritma Greedy 2
2.2. Cara Kerja Program 2
2.2.1. Mekanisme Aksi Bot 2
2.2.2. Implementasi Algoritma Greedy 2
2.2.3. Cara Menjalankan Bot 3
BAB III
APLIKASI STRATEGI GREEDY 4
3.1. Pemetaan Persoalan Battlecode 2025 4
3.2. Eksplorasi Alternatif Solusi 5
3.2.1. Alternatif 1: Coverage-Focused 5
3.2.2. Alternatif 2: Kill-Focused 5
3.2.3. Alternatif 3: Cost-Benefit 6
3.3. Analisis Efisiensi dan Efektivitas 6
3.3.1. Alternatif 1: Coverage-Focused 6
3.3.2. Alternatif 2: Kill-Focused 7
3.3.3. Alternatif 3: Cost-Benefit 7
3.4. Strategi Greedy Terpilih 7
BAB IV
IMPLEMENTASI DAN PENGUJIAN 9
4.1. Implementasi Alternatif Solusi 9
4.1.1. Alternatif 1: Coverage-Focused 9
4.1.2. Alternatif 2: Kill-Focused 12
4.1.3. Alternatif 3: Cost-Benefit 15
4.2. Implementasi utility Umum 18
4.3. Struktur Data dan Fungsi Bot 20
4.4. Pengujian Program 21
4.5. Analisis Hasil Pengujian 21
BAB V
KESIMPULAN DAN SARAN 22
5.1. Kesimpulan 22
5.2. Saran 22
LAMPIRAN 23
DAFTAR PUSTAKA 24

BAB 1
DESKRIPSI TUGAS

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot tim bekerja sama untuk mengalahkan tim lawan. Penulis akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain sesaing mungkin. Namun, hanya memiliki sumber daya komputasi terbatas per giliran, jadi harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi ***greedy*** dalam membuat bot ini. Berikut merupakan spesifikasi wajib untuk Tugas Besar pertama ini.

- Membuat 3 bot (1 utama dan 2 alternatif) dalam bahasa Java yang mengimplementasikan algoritma Greedy pada bot permainan Battlecode 2025 dengan tujuan memenangkan permainan;
- Strategi greedy yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memenangkan permainan dengan mewarnai peta sebanyak mungkin. Buatlah strategi greedy terbaik, karena setiap bot dari masing-masing kelompok akan diadu dalam kompetisi Tubes 1;
- Strategi greedy yang diimplementasikan harus berbeda untuk setiap bot yang diimplementasikan dan setiap strategi greedy harus menggunakan heuristic yang berbeda.

Terdapat juga spesifikasi bonus untuk Tugas Besar pertama sebagai berikut.

- Membuat video tentang aplikasi greedy pada bot serta simulasinya pada game.
- Memenangkan lomba kompetisi sesuai dengan posisi yang diraih.
- Membuat *post mortem analysis (an examination of a dead body to determine the cause of death)* secara mendetail yang mencakup kesalahan apa saja yang menyebabkan kekalahan dan kesalahan non-teknis lainnya. Selain itu, strategi mengembangkan bot kedepannya dan pelajaran yang didapatkan.

BAB II
LANDASAN TEORI

2.1. Algoritma Greedy

Algoritma *greedy* merupakan metode yang cukup sederhana, tetapi paling populer untuk memecahkan persoalan optimasi. Algoritma *greedy* adalah algoritma yang memecahkan persoalan secara langkah per langkah sehingga pada setiap langkahnya, dapat mengambil pilihan yang terbaik pada saat itu tanpa memperhatikan konsekuensi pemilihan tersebut ke depannya, dan berharap dengan memiliki pilihan optimum lokal pada setiap langkah akan berakhir dengan solusi optimum global.

Algoritma *greedy* bersifat Greedy Choice Property, yaitu solusi optimal dapat dibangun dengan memilih solusi optimum lokal terlebih dahulu; dan Optimal Substructure, yaitu solusi optimal dari persoalan dapat dibentuk dari potongan solusi untuk tiap sub-persoalannya. Prosedur *greedy* terdiri dari merumuskan persoalan optimasi dengan pilihan *greedy*, menunjukkan adanya *optimal substructure*, dan membuktikan adanya solusi *greedy-optimal*.

2.2. Cara Kerja Program

Program bot yang dikembangkan untuk Battlecode 2025 bekerja dalam *game loop* berbasis giliran (*turn-based*). Pada setiap gilirannya, masing-masing bot akan melakukan sebuah program berisi algoritma masing-masing sesuai dengan jenis bot yang dapat dibagi menjadi tiga tahapan utama: pembacaan kondisi sekitar, pengambilan keputusan, dan eksekusi sebuah aksi.

2.2.1. Mekanisme Aksi Bot

Bot melakukan aksi sesuai hasil pengamatan lingkungan di sekitarnya pada setiap giliran. Aksi sebuah bot dikendalikan oleh fungsi utama yang secara berulang akan mengevaluasi kondisi *map* sebagai berikut.

1. Memeriksa *health point*, *cooldown* aksi, dan sisa *paint*.
2. Mendeteksi posisi bot tim dan musuh, *terrain*, dan sumber daya sekitar.
3. Melakukan aksi sesuai penilaian dari hasil kondisi *map* sekitar seperti bergerak, transfer *paint* ke tower, maupun *skill* sesuai masing-masing bot.

2.2.2. Implementasi Algoritma Greedy

Algoritma *greedy* diimplementasikan pada tahap pengambilan keputusan. Bot tidak menyusun strategi jangka panjang atau bahkan memperhitungkan konsekuensi suatu pilihan untuk beberapa giliran ke depan, melainkan mencari sebuah pilihan optimal pada saat giliran itu juga. Berikut tahap implementasi algoritma *greedy*.

- Bot akan mengumpulkan seluruh aksi yang dapat dilakukan (himpunan kandidat) pada giliran tersebut.

- Fungsi *greedy* masing-masing bot akan mengevaluasi setiap kandidat menggunakan sebuah *heuristic* tertentu seperti jarak terdekat ke musuh atau tower, batas terluar dari area tim, atau perhitungan dampak sebuah *skill* jika diluncurkan ke sebuah target.
- Bot kemudian memilih aksi sesuai nilai paling optimal dan langsung mengeksekusinya di giliran tersebut tanpa menimbang konsekuensi untuk giliran berikutnya.

2.2.3. Cara Menjalankan Bot

Program dapat dijalankan menggunakan perintah bawaan dari *engine* permainan itu sendiri. Langkah-langkah untuk menjalankan program adalah sebagai berikut.

1. Pastikan *environment* dan *dependency* yang dibutuhkan sudah terinstal.
2. Buka terminal atau *command prompt* pada direktori utama folder.
3. Eksekusi perintah simulasi pertandingan dengan *./gradlew build*.
4. Pindah direktori ke *client* dan buka *file explorer*.
5. Jalankan aplikasi permainan dan sesuaikan pengaturan permainan yang ingin dipilih.
6. Jalankan program.

BAB III
APLIKASI STRATEGI GREEDY

3.1. Pemetaan Persoalan Battlecode 2025

Dalam pengimplementasian algoritma *greedy* pada bot, pencarian langkah terbaik untuk setiap bot (sesuai jenisnya) pada setiap giliran dapat dipetakan menjadi elemen-elemen algoritma *greedy* sebagai berikut.

- Himpunan Kandidat
  Seluruh kemungkinan aksi valid yang dapat dilakukan oleh bot pada satu giliran.
  Soldier: {Move, Attack, Self-Destruct}
  Mopper: {Move, Mop, Swing, Transfer, Self-Destruct}
  Splasher: {Move, Attack, Self-Destruct}

  Catatan: Move yang dimaksud adalah gerakan bot ke 8 arah mata angin, dan seluruh *skill* Attack, Mop, Swing, Transfer memiliki seluruh kemungkinan target atau pusat *tile*.

- Himpunan Solusi
  Kumpulan aksi terpilih dari himpunan kandidat yang telah dieksekusi bot sejak awal hingga akhir permainan atau sampai bot hancur.

- Fungsi Solusi
  Fungsi yang menentukan apakah permainan telah selesai (mencapai batas turn atau salah satu tim sudah menguasai 70% luas *map*).

- Fungsi Seleksi
  Fungsi yang memilih kandidat aksi yang valid terbaik pada giliran tersebut berdasarkan *heuristic* tertentu, seperti memilih target serangan dengan nilai paling optimal (sesuai fungsi objektif).

- Fungsi Kelayakan
  Fungsi yang memastikan sebuah kandidat aksi dapat dijalankan secara valid untuk giliran tersebut, seperti jika terdapat tembok di sisi utara bot, maka kandidat Move North tidak dapat dinilai menjadi sebuah solusi di giliran tersebut; atau ketika sebuah *skill* sedang *cooldown*, maka *skill* tersebut tidak dapat dinilai menjadi sebuah solusi di giliran tersebut.

- Fungsi Objektif
  Fungsi yang berfungsi untuk memaksimalkan skor tim (seperti pencarian target Splasher untuk *coverage* terbanyak) dan meminimalkan kerugian (seperti pencarian arah gerakan yang paling aman agar bot tidak kehabisan *paint*).

3.2. Eksplorasi Alternatif Solusi

Dalam Battlecode 2025, setiap jenis bot memiliki karakteristik yang unik dan peran masing-masing. Oleh karena itu, eksplorasi solusi *greedy* dilakukan dengan merumuskan 3 alternatif strategi/gaya bermain, di mana setiap strategi akan menggunakan *heuristic* yang berbeda untuk tiap jenis bot.

3.2.1. Alternatif 1: *Coverage-Focused*

Strategi ini berfokus pada syarat menang sebuah tim, yaitu menguasai 70% luas *map* secepat mungkin. Nilai heuristik dapat diprioritaskan pada penambahan *paint coverage*.

- Soldier
  Heuristik memprioritaskan pergerakan menuju area yang belum dicat atau menghapus cat musuh. Aksi Attack dinilai heuristik rendah karena tidak sebanding nilai *coverage* dengan biaya yang diperlukan, namun dapat dibutuhkan apabila terdapat musuh yang menghalangi jalan.
- Mopper
  Heuristik menilai Mop diprioritaskan untuk area yang dipenuhi cat musuh. Swing hanya digunakan apabila terdapat musuh yang berkumpul di dekatnya.
- Splasher
  Heuristik memprioritaskan pencarian *tile* target yang dapat menghasilkan *coverage* terbanyak.
- Tower
  Berfokus untuk menghasilkan Splasher sebanyak mungkin, karena Splasher adalah tipe robot yang sangat unggul dalam menimpa cat musuh.

3.2.2. Alternatif 2: *Kill-Focused*

Strategi ini bertujuan untuk melumpuhkan sumber daya dan pasukan musuh secepatnya, mengabaikan *coverage* dan terus memburu musuh.

- Soldier
  Heuristik dihitung berdasarkan jarak terdekat dan sisa HP musuh. Semakin rendah HP musuh di sekitarnya, semakin tinggi prioritas untuk Attack.
- Mopper
  Heuristik mencari nilai kepadatan musuh tertinggi di dalam jangkauan Swing.
- Splasher
  Heuristik memprioritaskan pencarian *tile* target yang memiliki nilai kepadatan musuh terbanyak agar memerlukan *paint* berlebih untuk berjalan. Heuristik akan menilai area terdampak yang mencakup Tower musuh sebagai nilai tertinggi.
- Tower
  Menghasilkan robot dengan mendahulukan tipe Soldier karena paling cocok dalam menyerang musuh sendiri.

3.2.3. Alternatif 3: *Cost-Benefit*

Strategi ini berfokus pada rasio *cost-benefit* dari penggunaan sumber daya yang dimiliki. Bot akan bertindak sangat efisien, tidak membuang *paint* untuk *coverage* yang tidak optimal, dan meminimalkan kematian bot.

- Soldier
  Dilakukan pengecekan sisa *paint* untuk menentukan apakah memprioritaskan untuk menyerang atau bergerak menuju area tim untuk pengisian ulang. Attack hanya dipilih jika rasio *damage* lebih tinggi dari *paint* yang diperlukan.
- Mopper
  Heuristik menilai Mop dapat dilakukan jika dapat membersihkan *tile* musuh dengan jumlah yang banyak sekaligus. Jika hanya menguntungkan untuk 1 *tile*, Mop akan diabaikan karena dinilai tidak efisien.
- Splasher
  Heuristik Splash hanya akan terpilih jika cakupannya 100% mengenai *tile* kosong atau musuh. Jangkauan yang tumpang tindih dengan cat tim sendiri dinilai sangat rendah karena tidak efisien.
- Tower
  Memprioritaskan dalam menghasilkan Soldier karena dapat mengecat *tile* secara efisien tanpa memperhitungkan seluruh kemungkinan titik pusat peluncuran *skill* seperti Splasher.

3.3. Analisis Efisiensi dan Efektivitas

Evaluasi kinerja dari ketiga alternatif solusi *greedy* yang telah dirumuskan dibutuhkan untuk memilih alternatif yang akan dijadikan sebagai solusi utama. Analisis ditinjau dari parameter efisiensi (penggunaan sumber daya) dan efektivitas (peluang memenangkan pertandingan).

3.3.1. Alternatif 1: *Coverage-Focused*

- Efisiensi
  Strategi ini rentang mengalami pemborosan sumber daya *paint*. Bot memprioritaskan terhadap *coverage*, sehingga dapat menghamburkan *paint* untuk melakukan *skill* yang menghasilkan sedikit *tile* daripada menjaga Tower yang sedang kritis.
- Efektivitas
  Sangat efektif pada fase awal permainan untuk memperluas wilayah dan membangun Tower dengan cepat. Namun, efektivitasnya menurun drastis saat berhadapan langsung dengan musuh karena bot akan mengabaikan strategi musuh demi mengecat *tile*.

3.3.2. Alternatif 2: *Kill-Focused*

- Efisiensi
  Gaya bermain paling boros sumber daya, karena melakukan serangan secara terus-menerus dan menghabiskan banyak giliran hanya untuk mengejar musuh yang sekarat. Jika serangan meleset atau bot mati duluan, tim kehilangan sumber daya secara percuma.
- Efektivitas
  Berisiko tinggi karena akan menghasilkan nilai efektivitas tinggi jika berhasil mengalahkan unit-unit penting musuh dengan cepat, sehingga merusak sistem bot lawan. Namun, strategi ini menjadi bernilai nol jika musuh bermain pasif dan fokus pada *coverage*, bot hanya akan membuang giliran untuk mengejar musuh dan gagal mencapai kemenangan.

3.3.3. Alternatif 3: *Cost-Benefit*

- Efisiensi
  Strategi ini merupakan yang paling hemat dan optimal dalam mengelola sumber daya dan HP bot. Setiap *paint* yang dikeluarkan selalu dihitung nilai manfaat yang dihasilkannya.
- Efektivitas
  Strategi yang paling stabil untuk menjaga agar tim bertahan hidup hingga akhir permainan. Kelemahannya berada pada pertumbuhan *coverage* yang lambat. Jika lawan berani melakukan strategi agresif, tim dapat kalah persentase *coverage* karena terlalu membatasi pengeluaran *paint* dan keseringan mundur untuk mengisi ulang sumber daya.

3.4. Strategi Greedy Terpilih

Berdasarkan eksplorasi dan alternatif terhadap tiga alternatif yang telah dirumuskan, strategi *greedy* yang dipilih untuk diimplementasikan sebagai solusi utama pada program bot adalah Alternatif 3: *Survival-Focused*. Pemilihan strategi ini didasarkan pada beberapa pertimbangan utama sebagai berikut.

1. Manajemen Sumber Daya yang Optimal
   Paint merupakan sumber daya terpenting pada permainan ini. Pemilihan aksi berdasarkan alternatif lain sering mengabaikan banyak *paint* yang dihabiskan. Dengan strategi efisiensi, fungsi seleksi selalu membandingkan antara hasil yang didapatkan dengan biaya yang dikeluarkan.
2. Keseimbangan Strategi
   Strategi ini secara tidak langsung menciptakan bot yang berperilaku dinamis. Hal ini mencegah tim kehilangan unit secara cuma-cuma hanya untuk mengejar *coverage* atau membunuh musuh.
3. Pencapaian Kemenangan yang Lebih Stabil

Sesuai syarat kemenangan, bot dengan strategi ini hanya akan mengeksekusi *skill* mereka jika target area benar-benar memberikan hasil persentase *coverage* yang signifikan. Bot tidak akan mengecat ulang area yang telah dicat oleh tim sehingga pertumbuhan *coverage* dilakukan secara efektif.

BAB IV
IMPLEMENTASI DAN PENGUJIAN

4.1. Implementasi Alternatif Solusi

4.1.1. Alternatif 1: *Coverage-Focused*

a. Tower

```
function runTower():
    // 1. Early Game Expansion
    if current_round == 1:
        spawnSoldiersInAllDirections()

    // 2. Defense and Combat
    if action_is_ready:
        // [Alt 1] towerAttack evaluates territory.
        // Uses AoE if swarmed (>= 3 enemies), otherwise greedily
        // targets enemies stepping on our allied paint.
        towerAttack()

    // 3. Economy & Upgrades
    if current_round >= UPGRADE_ROUND_THRESHOLD:
        upgradeSelf()

    // 4. Production (Resource Management)
    if current_chips >= BUILD_CHIP_RESERVE:
        // [Alt 1] greedySpawnUnit evaluates surrounding area.
        // Spawns Mopper if invaded, otherwise follows normal
spawn rotation.
        is_spawn_successful = greedySpawnUnit(spawnCount)

        if is_spawn_successful:
            spawnCount++
```

b. Soldier

```
function runSoldier():
    // 1. Ruin & Tower Building Logic
    unclaimed_ruin = findUnclaimedRuin()
    if unclaimed_ruin is found:
        markTowerBlueprint(unclaimed_ruin)

        // Find safe spot to paint the tower pattern
        target_tile = findNearestEmptyPatternTile(unclaimed_ruin)
        moveTowards(target_tile)

        // Paint the required pattern for the tower
        paintTowerPattern(unclaimed_ruin)
        completeTower(unclaimed_ruin)
        return // End turn

    // 2. State Machine: ATTACK vs ADVANCE
    enemy_tower = findEnemyTowerInRange()
```

```
    if enemy_tower is found OR current_state == ATTACK:
        current_state = ATTACK

        attackEnemyTower() // Move to and attack the tower
        greedyPaintAroundSelf() // [Alt 1] Prioritize enemy paint
while attacking

        if enemy_tower is destroyed:
            current_state = ADVANCE
        return // End turn

    // 3. Default State: ADVANCE (Frontier Expansion)
    current_state = ADVANCE

    // [Alt 1] Try to aggressively push into enemy territory
    is_exploring = advanceToFrontier()

    if not is_exploring:
        // Fallback: If no enemy paint found, follow walls and
keep painting
        direction = Pathfinding.wallFollow()
        if canMove(direction):
            move(direction)
            greedyPaintAroundSelf()
```

c. Splasher

```
function splashAttack():
    center_candidates = getTilesInAttackRadius()
    best_target = NULL
    max_score = 0

    for each center in center_candidates:
        if cannotAttack(center): continue

        current_score = 0

        // 1. Evaluate Enemy Units in the splash area
        for each enemy in enemies_around(center):
            if enemy is tower:
                current_score += TOWER_SCORE
            else if enemy is within inner_radius:
                current_score += ENEMY_ROBOT_SCORE

        // 2. Evaluate Territory (Greedy Counter-Painting
Heuristic)
        for each tile in tiles_around(center):
            if tile has enemy paint and is within inner_radius:
                current_score += (ENEMY_PAINT_SCORE * 3) // High
multiplier for territory steal
            else if tile is empty:
                current_score += EMPTY_PAINT_SCORE
            else if tile has allied paint:
                current_score -= PENALTY_SCORE // Avoid wiping
our own paint
```

```
        // Selection
        if current_score > max_score:
            max_score = current_score
            best_target = center

    // Execute the most optimal AoE attack
    if best_target != NULL:
        attack(best_target)


function moveToEnemy():
    visible_enemies = getVisibleEnemies()
    target_location = NULL

    // Priority 1: Focus on enemy towers
    for each enemy in visible_enemies:
        if enemy is tower:
            target_location = enemy.location
            break

    // Priority 2: Fallback to any visible enemy robot
    if target_location == NULL and visible_enemies is not empty:
        target_location = visible_enemies[0].location

    // Execute Movement
    if target_location != NULL:
        direction = Pathfinding.findPathTo(target_location)
        if direction is valid and passable:
            move(direction)
    else:
        // Priority 3: No enemies visible, explore for territory
        direction = Pathfinding.findUnpaintedDirection()
        if direction is valid and passable:
            move(direction)
```

d. Mopper

```
function mopperAction():
    // Priority 1: Mop enemy territory (Greedy Counter-Painting)
    best_mop_target = findTileWithMostEnemyPaint()

    if best_mop_target != NULL:
        attack(best_mop_target) // Executes mop
        return

    // Priority 2: Swing at enemy robots
    enemy_target = findEnemyWithMostPaint()

    if enemy_target != NULL:
        attack(enemy_target) // Executes swing
        return

    // Priority 3: Support allies as a last resort
    ally_target = findAllyWithLowestPaint()
```

```
    if ally_target != NULL and hasEnoughPaint():
        transferPaint(ally_target)

function moveToPerimeter():
    best_direction = NULL
    max_score = -INFINITY

    for each direction in available_directions:
        if cannotMove(direction): continue

        current_score = 0
        target_location = currentLocation + direction

        // Penalty to encourage leaving allied territory
        if target_location has allied paint:
            current_score -= PENALTY_SCORE

        // Heuristic: Evaluate tiles around the potential next
step
        for each tile around target_location:
            if tile has enemy paint:
                current_score += (BORDER_SCORE * 4) // Huge
multiplier to dive into enemy paint
            else if tile is empty:
                current_score += (BORDER_SCORE / 2)

        // Greedy selection
        if current_score > max_score:
            max_score = current_score
            best_direction = direction

    // Execute movement
    if best_direction != NULL:
        move(best_direction)
    else:
        // Fallback: Explore towards unpainted areas if no enemy
paint is found
        explore_direction = Pathfinding.findUnpaintedDirection()
        if explore_direction is valid:
            move(explore_direction)
```

4.1.2. Alternatif 2: *Kill-Focused*

a. Tower

```
function runTower():

    // 1. Combat Priority
    if action_is_ready:
        // find the enemy with the lowest HP within attack range
        target = Sensing.findNearestLowestHP()

        if target != null:
            if canAttack(target.location):
```

```
                attack(target.location)

    // 2. Defensive Awareness
    enemy_count = countNearbyEnemies()

    if enemy_count >= 3:
        // use area attack when many enemies are nearby
        performAreaAttack()

    // 3. Economy and Upgrades
    if current_round >= UPGRADE_ROUND_THRESHOLD:
        upgradeSelf()

    // 4. Unit Production
    if current_chips >= BUILD_CHIP_RESERVE:

        // prioritize combat units when enemies are detected
        if enemy_count > 0:
            spawn Soldier
        else:
            spawn Splasher
```

b. Soldier

```
function runSoldier():

    // 1. Search for enemy targets
    enemy = Sensing.findNearestLowestHP()

    if enemy != null:

        // attack if the enemy is within range
        if canAttack(enemy.location):
            attack(enemy.location)
            return

        // move toward the detected enemy
        direction = Pathfinding.pathfind(enemy.location)

        if direction != null:
            move(direction)

        return

    // 2. Move toward enemy territory (enemy paint)
    enemy_paint_tile = Sensing.findEnemyPaint()

    if enemy_paint_tile != null:

        direction =
Pathfinding.pathfind(enemy_paint_tile.location)

        if direction != null:
            move(direction)
```

```
        return

    // 3. Explore unpainted territory
    direction = Pathfinding.exploreUnpainted()

    if direction != null:
        move(direction)
```

c. Splasher

```
function runSplasher():

    // 1. Prioritize area attacks on groups of enemies
    enemy_group = detectEnemiesInAttackRadius()

    if enemy_group not empty:

        // choose the best attack location that hits the most
enemies
        best_target = chooseBestAreaTarget(enemy_group)

        if canAttack(best_target):
            attack(best_target)
            return

    // 2. Move toward enemy paint to pressure enemy territory
    enemy_paint_tile = Sensing.findEnemyPaint()

    if enemy_paint_tile != null:

        direction =
Pathfinding.pathfind(enemy_paint_tile.location)

        if direction != null:
            move(direction)

        return

    // 3. Explore unpainted areas
    direction = Pathfinding.exploreUnpainted()

    if direction != null:
        move(direction)
```

d. Mopper

```
function runMopper():

    // 1. Remove enemy paint from the map
    enemy_paint_tile = Sensing.findEnemyPaint()

    if enemy_paint_tile != null:

        if canAttack(enemy_paint_tile.location):
```

```
            attack(enemy_paint_tile.location)
            return

        direction =
Pathfinding.pathfind(enemy_paint_tile.location)

        if direction != null:
            move(direction)

        return

    // 2. Support map control by exploring unpainted tiles
    direction = Pathfinding.exploreUnpainted()

    if direction != null:
        move(direction)
```

4.1.3. Alternatif 3: *Cost-Benefit*

a. Tower (Tower.java and RobotPlayer.java, dengan angka menggantikan nilai variabel di Constants.java)

```
function runTower():
    # 1. Opening (round 1)
    if round == 1:
        # force spawn soldiers on the 1st round
        for each direction around the tower:
            spawn Soldier if possible

    # 2. Defense
    if action is ready:
        target = find nearest enemy with the lowest HP
        if target found:
            single_target_attack(target)

        # AoE defense if swarmed
        enemies_in_range = count nearby enemies
        if enemies_in_range >= 3:
            AoE_attack()

    # 3. Economy
    if round >= 200:
        upgrade_self_to_level_2_or_3()

    # 4. Spawning units (only if we have enough chips)
    if chips >= 1200:
        # ratio 3 Soldier : 1 Mopper : 1 Splasher
        unit_type = type_next_cycle()
        spawn_unit(unit_type)
```

b. Soldier (Soldier.java and RobotPlayer.java, dengan angka menggantikan nilai variabel di Constants.java)

```
function runSoldier():
```

```
    # 1. Retreat if low
    if health < 25% or paint < 20%:
        retreat()
        return

    # 2. Tower Building / Ruins
    closest_ruin = find_nearest_unclaimed_ruin()
    if closest_ruin is found:
        if can see the ruin:
            # 'Random' function but same for each coordinate
            if hash(ruin.x, ruin.y) % 2 < 1:
                mark_ruin_for_Paint_Tower()
            else:
                mark_ruin_for_Money_Tower()

            move_towards_an_unpainted_tile_in_the_ruin_pattern()
            paint_the_pattern_tiles_we_are_standing_on()

            attempt_to_complete_tower_build()
        else:
            move_towards(closest_ruin)
        return

    # 3. Attack
    enemy_tower = detect_enemy_tower()
    if enemy_tower found or currently in ATTACK state:
        state = ATTACK
        move_towards_and_attack(enemy_tower)
        paint_tiles_around_our_feet()

        if enemy tower is destroyed:
            state = ADVANCE
        return

    # 4. Explore(default)
    state = ADVANCE
    if cannot explore unpainted tiles:
        # If stuck, hug the walls until free
        follow_walls()
        paint_tiles_around_our_feet()
```

c. Splasher (Splasher.java dengan angka menggantikan nilai variabel di Constants.java)

```
function runSplasher():
    # ACTION PHASE
    if action is ready and paint >= 50:
        best_target = None
        max_score = 0

        # Test every single tile in our attack radius as the
center of an AoE attack
        for each tile in attack_radius:
            score = 0
```

```
            # AoE attacks hit a wide radius
            if enemy_tower in splash_zone:
                score += 1000
            if enemy_robot in inner_splash_zone:
                score += 20
            if enemy_paint in inner_splash_zone:
                score += 5
            if empty_paint in splash_zone:
                score += 2

            if score > max_score:
                max_score = score
                best_target = tile

        if best_target found:
            AoE_Attack(best_target)

    # MOVEMENT PHASE
    if movement is ready:
        enemy_target = find_nearest_enemy() #prioritize tower

        if enemy_target found:
            move_towards(enemy_target)
        else:
            explore_unpainted_territory()
```

d. Mopper (Mopper.java dengan angka menggantikan nilai variabel di Constants.java)

```
function runMopper():
    # ACTION PHASE
    if action is ready:
        # Give paint away if we have enough and an ally is
starving
        starving_ally =
find_adjacent_ally_with_less_than_50_paint()
        if our_paint >= 30 and starving_ally exists:
            transfer_20_paint_to(starving_ally)

        # Otherwise, attack the enemy with the highest paint
        else:
            highest_paint_enemy =
find_adjacent_enemy_with_most_paint()
            if highest_paint_enemy exists:
                attack(highest_paint_enemy)

    # MOVEMENT PHASE
    if movement is ready:
        # Global pathfinding: if we see a starving ally far away,
walk to them
        if our_paint >= 30:
            distant_starving_ally =
find_ally_in_vision_with_less_than_50_paint()
            if distant_starving_ally is far away:
```

```
                move_towards(distant_starving_ally)
                return

        # Greedy perimeter movement (protecting the borders of
our paint)
        best_direction = None
        max_score = -Infinity

        # Scoring
        for each possible direction (N, S, E, W, etc.):
            score = 0
            if direction has enemy paint:
                score = -Infinity
            if direction has allied paint:
                score += 10
            if direction borders empty/enemy paint:
                score += 5
            if direction gets us next to a starving ally:
                score += 50

            if score > max_score:
                max_score = score
                best_direction = direction

        move(best_direction)
```

4.2. Implementasi utility Umum

a. Pathfinding

```
function pathfind(target_location):
    min_distance = Infinity
    best_paint_cost = None
    best_location = None

    for each square we can walk on:
        distance = calculate_distance_to(target_location)
        paint_type = get_paint_on_square()

        if distance < min_distance and we_have_paint_to_step_on_it:
            min_distance = distance
            best_paint_cost = paint_type
            best_location = square

        else if distance == min_distance:
            if square is cheaper to step on than best_paint_cost:
                best_paint_cost = paint_type
                best_location = square

    return direction to best_location

function exploreUnpainted():
    tiles = get_moveable_tiles()

    if tiles is empty:
```

```
        # If surrounded by paint, try to just move anywhere
        empty_tiles = get_all_valid_moves()

    if stuck:
        return follow_walls_algorithm() # Turn 45 degrees right until
not wall

    best_tile = tile in empty_tiles that has the most empty space behind
it
    return direction to best_tile
```

b. Sensing

```
function getGreedyDirectionToward(target):
    best_direction = None
    lowest_score = Infinity

    for each adjacent square we can walk to:
        # score based on distance + the paint on the tile
        score = distance_to(target) * 10

        paint_on_square = get_paint(square)
        if paint_on_square == ENEMY:
            score += 8
        else if paint_on_square == EMPTY:
            score += 2
        else if paint_on_square == ALLY:
            score -= 2

        if score < lowest_score:
            lowest_score = score
            best_direction = direction

    return best_direction

function findNearestLowestHP():
    weakest_enemy = None
    lowest_hp = Infinity

    for each enemy in attack_radius:
        if enemy.HP < lowest_hp:
            lowest_hp = enemy.HP
            weakest_enemy = enemy

    return weakest_enemy

function canBuildTower(ruin_location):
    for each tile in the 5x5 area around the ruin:
        if tile is the center (the ruin itself):
            if there is a robot standing on the ruin:
                return False #can't build right now
        else:
            if tile is NOT painted with our Allied blueprint color:
                return False #pattern is incomplete or sabotaged by
enemy
```

```
    return True
```

c. Robot

```
function hasLowPaint(percentage):
    if current_paint < (max_paint_capacity * percentage / 100):
        return True
    return False

function hasLowHealth(percentage):
    if current_health < (max_health * percentage / 100):
        return True
    return False

function markTower(tower_type, target_location):
    square_to_mark = target_location - direction_to(target_location)
    if square_to_mark is not marked yet:
        place_tower_blueprint(tower_type, target_location)

function completeTower(target_location):
    if can_build(MONEY_TOWER):
        build(MONEY_TOWER)
    if can_build(PAINT_TOWER):
        build(PAINT_TOWER)
    if can_build(DEFENSE_TOWER):
        build(DEFENSE_TOWER)
```

4.3. Struktur Data dan Fungsi Bot

Algoritma *greedy* utama pada bot digunakan untuk menentukan arah gerakan menuju target. Fungsi yang bertanggung jawab terhadap proses ini adalah *getGreedyDirectionToward* dan *scoreDirection*. Fungsi *getGreedyDirectionToward* mengevaluasi seluruh arah yang mungkin dari posisi robot saat ini. Setiap arah yang memungkinkan untuk dilalui akan dievaluasi menggunakan fungsi *scoreDirection*. Fungsi ini menghitung nilai *score* berdasarkan jarak menuju target dan kondisi *tile* tujuan. Nilai *score* dihitung dengan rumus jarak kuadrat menuju target dikalikan suatu bobot, kemudian ditambahkan penalti atau bonus berdasarkan jenis paint pada tile tersebut. Jika *tile* memiliki *enemy paint* maka diberikan penalti yang besar karena langkah tersebut merugikan robot. Jika tile kosong maka diberikan penalti kecil karena pergerakan akan mengurangi paint robot. Jika tile memiliki *ally paint* maka diberikan bonus karena pergerakan tersebut aman dan tidak menyebabkan kehilangan *paint*. Arah yang memiliki nilai *score* paling kecil akan dipilih sebagai arah gerakan terbaik karena dianggap memberikan keuntungan paling besar secara lokal.

Prosedur pergerakan robot diimplementasikan pada fungsi *moveGreedyToward*. Prosedur ini pertama memeriksa apakah robot masih memiliki kemampuan untuk bergerak pada giliran tersebut. Jika robot masih dapat bergerak maka fungsi akan memanggil

getGreedyDirectionToward untuk menentukan arah terbaik menuju target. Apabila arah yang dihasilkan valid dan robot dapat bergerak ke arah tersebut maka robot akan melakukan aksi pergerakan. Prosedur ini memastikan bahwa robot selalu mengambil langkah yang memiliki nilai evaluasi terbaik menurut fungsi *greedy*.

4.4. Pengujian Program

Pengujian dilakukan menggunakan client viewer sebanyak 3 kali antar bot alternatif dengan jumlah 75 peta yang tersedia di permainan. Berikut merupakan hasil pertandingan setiap pertandingan bot.

Coverage-Focused	vs	*Kill-Focused*
58		17
Coverage-Focused	vs	*Cost-Benefit*
33		42
Kill-Focused	vs	*Cost-Benefit*
16		59

Dari hasil pengujian juga didapatkan *Cost-Benefit* unggul pada peta yang memiliki banyak rintangan, *Coverage-Focused* unggul pada peta yang luas dan tidak memiliki rintangan, serta *Kill-Focused* unggul pada peta berukuran kecil karena dapat langsung mematikan tower musuh secara cepat.

4.5. Analisis Hasil Pengujian

Dari hasil pengujian, didapatkan bahwa Alternatif 3: *Cost-Benefit* memenangkan 2 set pertandingan dari bot alternatif lainnya. Oleh karena itu, sesuai analisis dan pengujian, Alternatif 3 merupakan solusi utama yang akan kami pilih dan merupakan heuristik terbaik untuk Battlecode 2025.

BAB V
KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian algoritma *greedy* pada Battlecode 2025, dapat dirumuskan beberapa kesimpulan sebagai berikut.

1. Algoritma *greedy* berhasil diaplikasikan pada penentuan aksi setiap bot pada setiap gilirannya. Bot dapat mengambil keputusan lokal paling optimal berdasarkan heuristik yang dipilih tanpa memperhitungkan konsekuensi pada giliran selanjutnya.
2. Dari tiga alternatif strategi yang dieksplorasi dan dianalisis, strategi *Cost-Benefit* dipilih menjadi solusi yang paling optimal untuk diimplementasikan. Strategi ini berfokus pada menjaga stabilitas tim dengan menyeimbangkan rasio antara sumber daya yang dikeluarkan dan area yang didapat.
3. Berdasarkan hasil pengujian simulasi pertandingan, algoritma *greedy* ini berhasil memenangkan mayoritas pertandingan pada kondisi musuh yang berfokus pada dominasi dan juga agresif di seluruh peta yang tersedia pada permainan. Hal ini juga menandakan bahwa algoritma tersebut merupakan salah satu heuristik terbaik yang secara stabil memenangkan pertandingan Battlecode 2025.

5.2. Saran

Saran yang dapat kami berikan untuk Tugas Besar 1 Strategi Algoritma adalah sebagai berikut.

1. Dapat membagikan *repository* Github pemenang perlombaan Battlecode sebelumnya yang mengimplementasikan algoritma Greedy agar dapat dijadikan referensi.
2. Sangat banyak fitur atau metode yang dapat dilakukan di Battlecode 2025. Namun, karena keterbatasan waktu dan sumber daya, penggunaan fitur menjadi kurang maksimal.
3. Dapat dibuat media *sparring* untuk menyediakan uji coba tim terhadap seluruh tim, dengan menjaga kerahasiaan *source code*, hal ini dapat mendukung sifat kompetitif dan rasa ingin meningkatkan performa algoritma secara terus menerus.

LAMPIRAN

Tautan Github: https://github.com/IzzyMeow/Tubes1_AcademyOfFineArts

Tautan Youtube: https://youtu.be/0xyQUTHfieg

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

(Bernhard Aprillio Pramana)

(Moreno Syawali Ganda Sugita)

(Nathaniel Christian)

DAFTAR PUSTAKA

Munir, Rinaldi. Strategi Algoritma 2025-2026. STEI ITB. informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.htm, diakses pada 8 Maret 2026 pada pukul 22.40